

Lifting E-Graphs: A Function Isn't A Constant

Philip Zucker

EGRAPHS 2026

The Plan

1. A Semantics for Variables I Like
2. Lifting & Thinning
3. Lifting E-Graphs
 - i. Lift Pulling Smart Constructors
 - ii. Thinning Union Find

Original Sin

$\sin(x)$

Explicit Names Kind of Suck

1. Runaway generation of names during eqsat

- $P \rightarrow \forall x, P$ where x is fresh

2. Scope Hygiene

- $x * 0 = 0 \implies \text{lam}(y, 0) = \text{lam}(y, x * 0)$ 😬

3. Not enough sharing

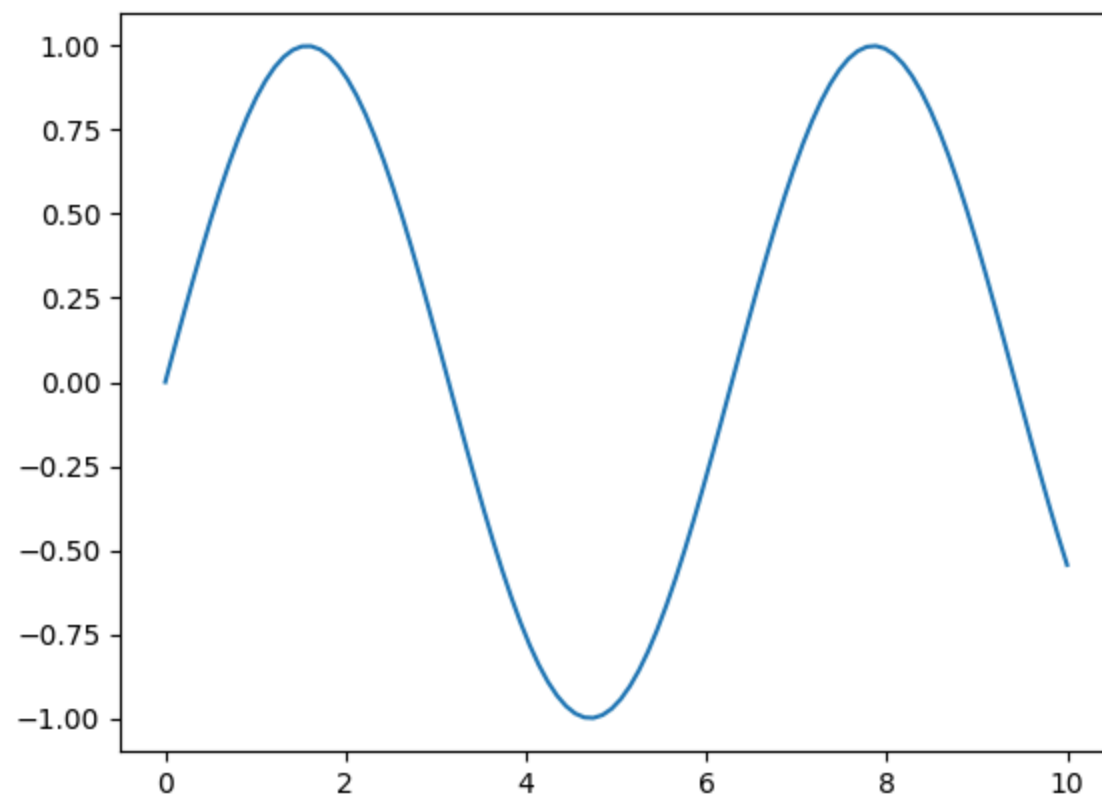
- `f(g(h(x)))` and `f(g(h(y)))` share no memory

4. ***Too much sharing...***

What is $\sin(x)$?

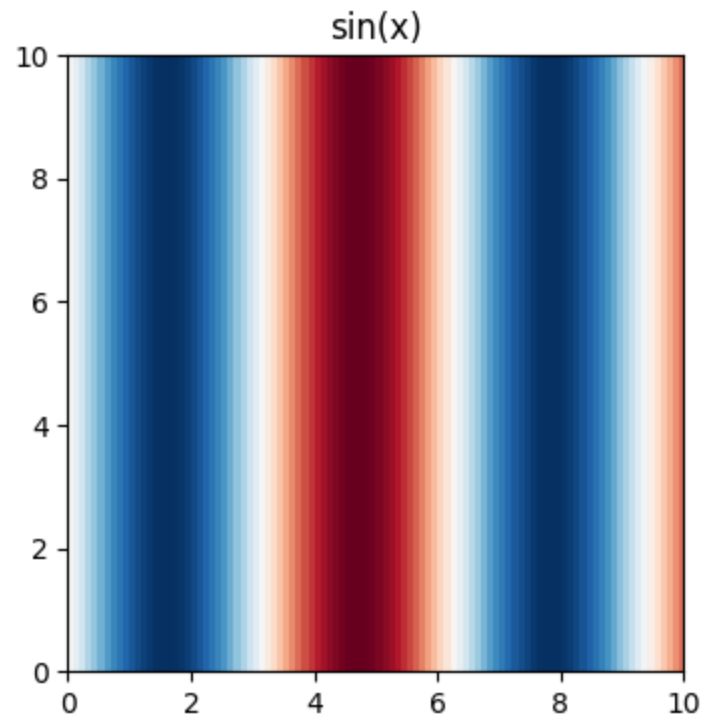
Is It This?

$$x \mapsto \sin(x) : \mathbb{R}^1 \rightarrow \mathbb{R}$$



Or This?

$$x, y \mapsto \sin(x) : \mathbb{R}^2 \rightarrow \mathbb{R}$$



Breaking the Ambiguity

Design Principle

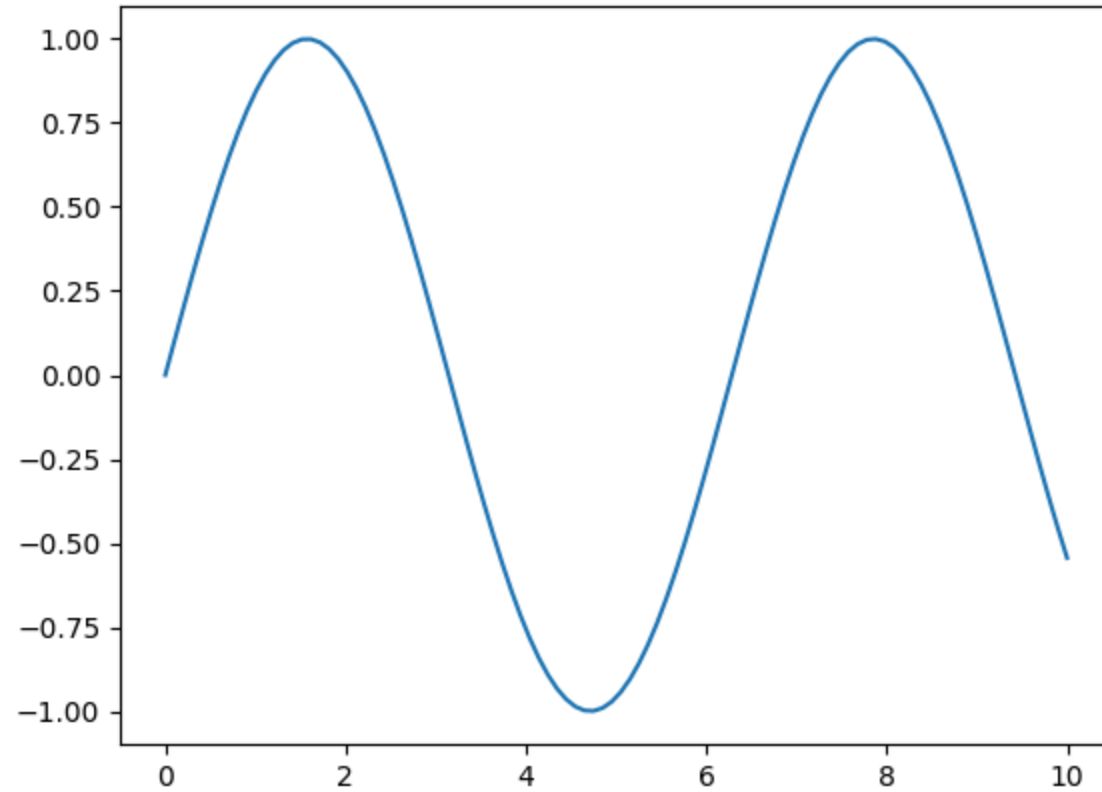
The Context $x, y \mapsto _$ is
not *where* a term is,
it is a part of *what* a term is

Combinatorizing $x, y \mapsto _$

- Naive Well-Dimensioned Nameless Representation
- Index everything on dimension/context d
 - $42_d, \sin_d, +_d$
- Refer to variable by index
 - var_{di} - variable i in dimension/context d
- Usable in ordinary e-graph

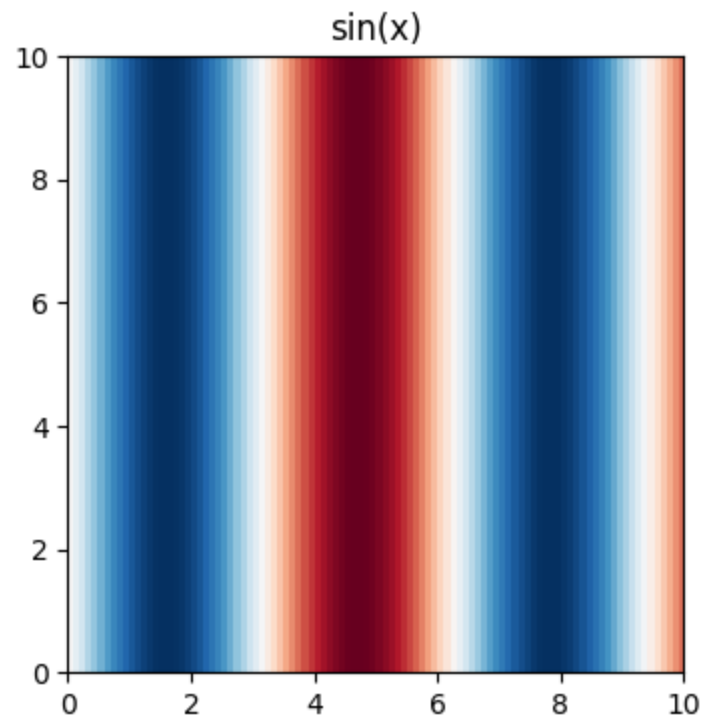
1-D

$$x \mapsto \sin(x) : \mathbb{R}^1 \rightarrow \mathbb{R} \implies \sin_1(\text{var}_{10})$$



2-D

$$x, y \mapsto \sin(x) : \mathbb{R}^2 \rightarrow \mathbb{R} \implies \sin_2(\text{var}_{20})$$



A Compositional Pointwise Semantics

- $[[42_d]] = v_0, v_1, \dots, v_{d-1} \mapsto 42$
 - The *function* $[[42_d]] : \mathbb{R}^d \rightarrow \mathbb{R}$ is *not* the *constant* 42.
- $[[\mathbf{var}_{di}]] = v_0, v_1, \dots, v_{d-1} \mapsto v_i$
 - $\mathbf{var}_{di}$ is a *projection* of type $\mathbb{R}^d \rightarrow \mathbb{R}$
- $[[\sin_d(t)]] = v_0, v_1, \dots, v_{d-1} \mapsto \sin([t](v_0, v_1, \dots, v_{d-1}))$
 - $[[\sin_d]]$ is a *higher order function* of type $(\mathbb{R}^d \rightarrow \mathbb{R}) \rightarrow (\mathbb{R}^d \rightarrow \mathbb{R})$
 - $\sin_d \sim \sin \cdot _$
- $[[t +_d s]] = v_0, v_1, \dots, v_{d-1} \mapsto [t](v_0, v_1, \dots, v_{d-1}) + [s](v_0, v_1, \dots, v_{d-1})$
- and so on...

Is This Good?

- 😊 $\sin(x)$ semantic ambiguity gone
- 😞 Even *more* very similar entities that share no memory
 - $\sin_1(\text{var}_{10}) : \mathbb{R}^1 \rightarrow \mathbb{R}$ and $\sin_2(\text{var}_{20}) : \mathbb{R}^2 \rightarrow \mathbb{R}$ are not *equal* but are clearly *related*
 - In what sense?

Lifting Functions

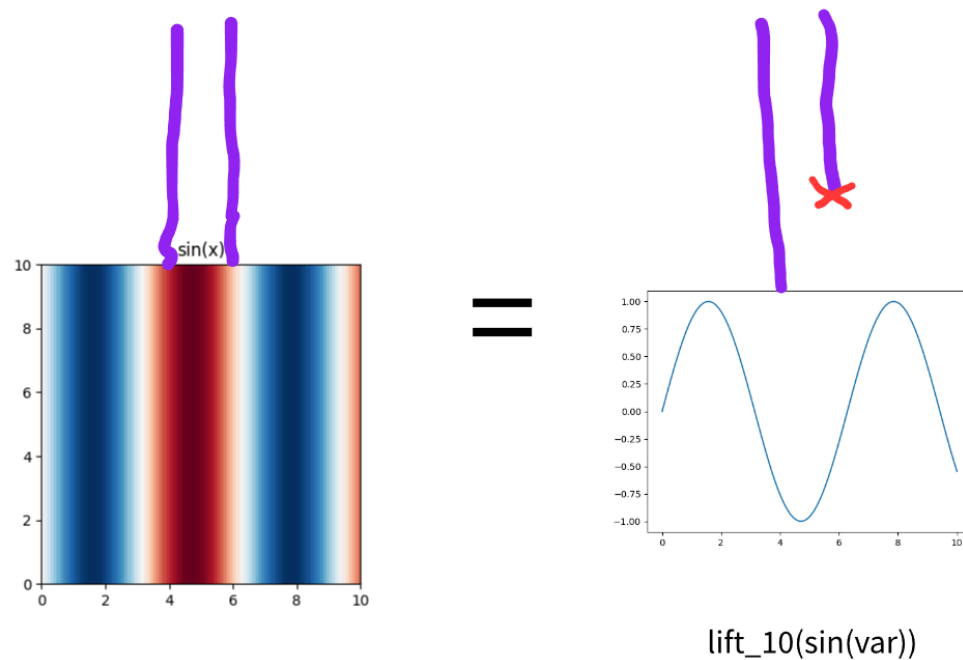
- Lift = Keep some args and throw some away

```
def lift(thin : list[bool], f):  
    return lambda *args: f(*[arg for keep, arg in zip(thin, args) if keep])
```

- $\text{lift}_t : (\mathbb{R}^{\text{popcnt}(t)} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R}^{\text{len}(t)} \rightarrow \mathbb{R})$
- t is a **thinning** bitvector (1=keep, 0=drop)
 - Ex: $\text{lift}_{101} = \text{keep arg0, drop arg1, keep arg2}$

Lifting $x \mapsto \sin(x)$

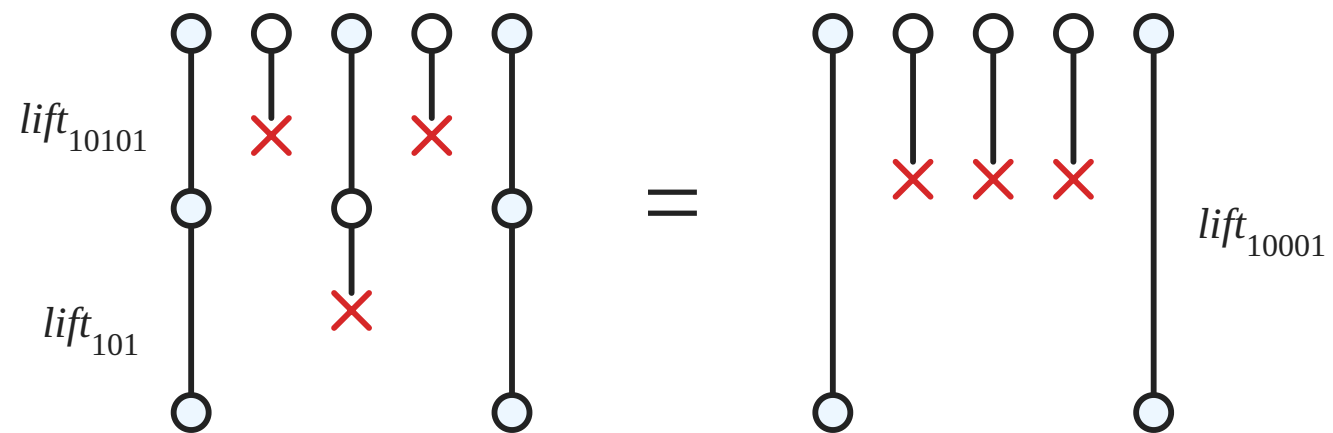
$$\begin{aligned}\sin_1(\text{var}_{10}) &:= \sin(\text{var}) \\ \sin_2(\text{var}_{20}) &:= \text{lift}_{10}(\sin(\text{var}))\end{aligned}$$



Thinnings: Simple and Underappreciated

- One *lifts* functions by *thinning* its arglist
- Thinnings are
 - strictly monotonic maps between totally ordered finite sets
 - recipe to extract subsequences
- ~ compaction of multiple de Bruijn shifts
- more minimalist than permutations

Thinning Composition



Lifting Rules

Compose / Constant Fold.	$\forall x, \text{lift}_i(\text{lift}_j(x)) = \text{lift}_{i \cdot j}(x)$
Pulling / Homomorphism	$\forall xy, f(\text{lift}_t(x), \text{lift}_t(y)) = \text{lift}_t(f(x, y))$

Lifting E-Graphs

- Bake Lift In
- E-Graph Modulo Theories (EMT)
 - Recipe: replace `Id` with `FatId` everywhere

```
type FatId = (list[bool], int)
```

Lift-Pulling Smart Constructors

```
class ENode:  
    f : str  
    args : list[FatId]
```

- $f(\text{lift}_i(X), \text{lift}_i(Y)) \rightarrow \text{lift}_i(f(X, Y))$
- Factor out needed vars
 - bitwise or `|`
 - min context ~ free variable analysis

Examples

Named Form	Unnormalized	Max Pulled
$x, y, z, w \mapsto 42$	$\text{lift}_{0000}(42)$	$\text{lift}_{0000}(42)$
$x, y, z, w \mapsto 42 + 42$	$\text{lift}_{0000}(42) + \text{lift}_{0000}(42)$	$\text{lift}_{0000}(42 + 42)$
$x, y \mapsto 42 + 42$	$\text{lift}_{00}(42) + \text{lift}_{00}(42)$	$\text{lift}_{00}(42 + 42)$
$x, y, z, w \mapsto z + 42$	$\text{lift}_{0010}(\text{var}) + \text{lift}_{0000}(42)$	$\text{lift}_{0010}(\text{var} + \text{lift}_0(42))$

Thinning Union Find

- `type UnionFind = list[FatId]`
- Like Offset (Group) Union Find
 - $e_8 - 3 = e_3 + 11 \implies e_8 \rightarrow e_3 + 14$
 - `uf[8] == (14, 3)`
- Find
 - Compose $e_{child} \rightarrow \text{lift}_t(e_{parent})$
- Union
 - Peel off common lift (lift is injective)
 - Parent is sometimes forced / requires make-set
 - lift_t is *not* invertible
 - Semantics: Learn directions are constant

Typical Example

Named Form	$(x, y \mapsto y * 1) = (x, y \mapsto y)$
Pulled Form	$\text{lift}_{01}(\text{var} * \text{lift}_0(1)) = \text{lift}_{01}(\text{var})$
Interned Form	$\text{lift}_{01}(e_{17}) = \text{lift}_{01}(e_{42})$
Peel	$e_{17} = e_{42}$
Orient	$e_{17} \rightarrow e_{42}$ (or $e_{42} \rightarrow e_{17}$)

Redundancy Example: Parent Forced

Named Form	$(x \mapsto x * 0) = (x \mapsto 0)$
Pulled Form	$\text{var} * \text{lift}_0(0) = \text{lift}_0(0)$
Interned Form	$e_{92} = \text{lift}_0(e_8)$
Orient (forced)	$e_{92} \rightarrow \text{lift}_0(e_8)$

E-matching

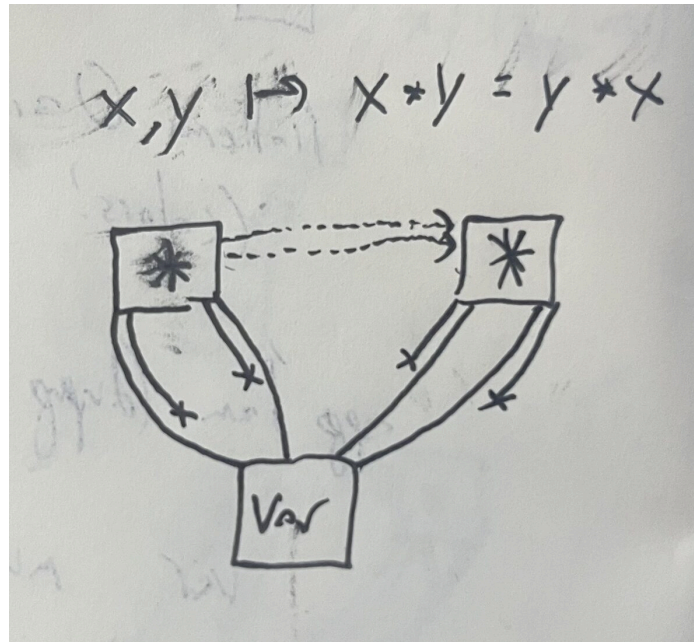
- Typical case is simple
 - Accumulate thinnings as you traverse into terms
 - *push* lifts $\text{lift}_t(f(e_1, e_2)) \rightarrow f(\text{lift}_t(e_1), \text{lift}_t(e_2))$
- Redundancies in union find $e_{\text{child}} \rightarrow \text{lift}_k(e_{\text{parent}})$ are trickier
 - Choice 1: Don't Match Through Them
 - Choice 2: Factor $\text{lift}_i(e_{\text{parent}}) = \text{lift}_{?j}(\text{lift}_k(e_{\text{parent}}))$
 - Multiple solutions
 - Ex: $(x, y \mapsto 0) =_? (x, y \mapsto ?a * 0)$
 - solutions $?a = x$ and $?a = y$

Related Work

- Slotted E-Graphs (Thanks Rudi Schneider!)
- McBride's Everybody's Got to Be Somewhere (Co de Bruijn)
- Semantics of type judgements $[[x : \mathbb{R}, y : \mathbb{R} \vdash \sin(x) : \mathbb{R}]] \in \mathbb{R}^2 \rightarrow \mathbb{R}$
- Explicit Weakening Calculi
- de Bruijn Succ Floating

Questions?

Blog Posts https://www.philipzucker.com/lifting_egraph/



$$\text{lift}_{10}(\text{var}) * \text{lift}_{01}(\text{var}) \rightarrow \text{lift}_{01}(\text{var}) * \text{lift}_{10}(\text{var})$$

Questions?

Blog Posts https://www.philipzucker.com/lifting_egrph/

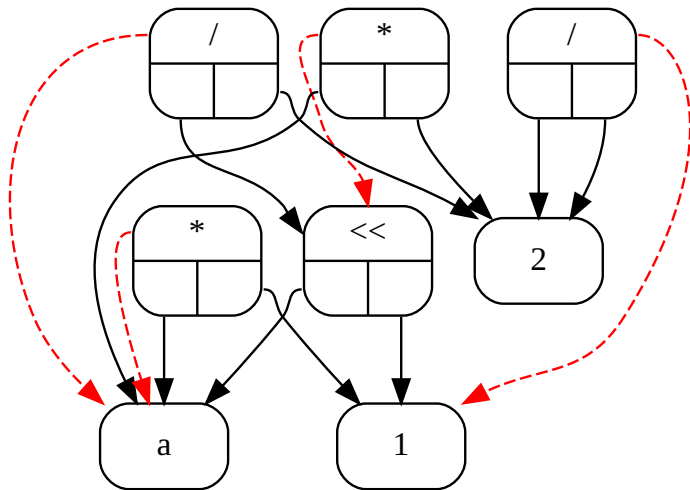
1. Problems with Named Variables
2. A Semantics for Variables I like
3. Naive Nameless
4. Lifting & Thinning
5. Lifting E-Graphs
 - i. Smart Constructors
 - ii. Thinning Union Find
 - iii. E-matching
6. Related Work

Questions?

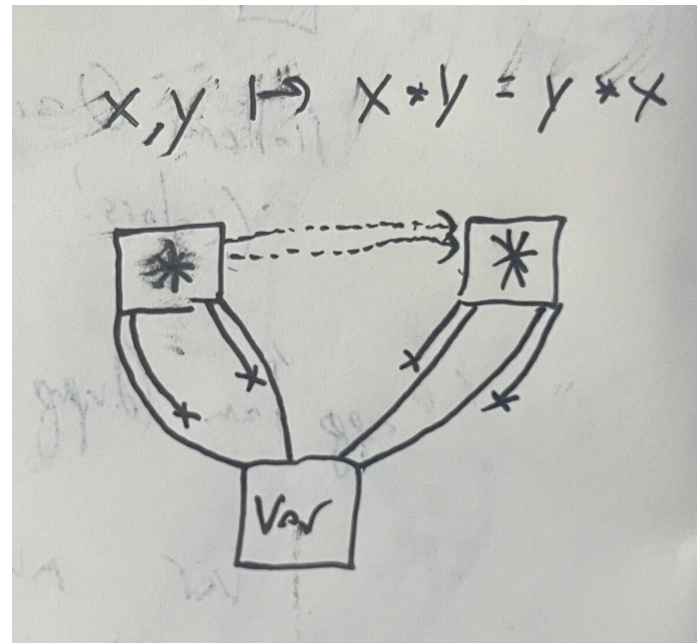
Visualizing

- Thicken all e-graph edges to be thinnings

Regular E-Graph



Lifting E-Graph



$$\text{lift}_{10}(\text{var}) * \text{lift}_{01}(\text{var}) \rightarrow \text{lift}_{01}(\text{var}) * \text{lift}_{10}(\text{var})$$

E-matching

- Accumulate thinnings as you traverse into terms

$$\begin{array}{ll} \text{lift}_i(e_1) =_? f(?a, ?b) & \text{goal} \\ \text{lift}_i(f(e_2, e_3)) =_? f(?a, ?b) & \text{nondet e-class expand } e_1 \rightarrow f(e_2, e_3) \\ f(\text{lift}_i(e_2), \text{lift}_i(e_3)) =_? f(?a, ?b) & \text{lift-pushing} \\ \Rightarrow ?a = \text{lift}_i(e_2) \quad \wedge \quad ?b = \text{lift}_i(e_3) & \text{decompose } f \end{array}$$

- Redundancies in union find $e_{\text{child}} \rightarrow \text{lift}_k(e_{\text{parent}})$ are trickier
 - Choice 1: Don't Match Through Them
 - Choice 2: Factor $\text{lift}_i(e_{\text{parent}}) = \text{lift}_{?j}(\text{lift}_k(e_{\text{parent}}))$
 - Multiple solutions
 - Ex: $(x, y \mapsto 0) =_? (x, y \mapsto ?a * 0)$
 - solutions $?a = x$ and $?a = y$

Redundancy Example 2: Incompatible liftings

Named Form	$(x, y \mapsto x * 0) = (x, y \mapsto 0 * y)$
Pulled Form	$\text{lift}_{10}(\text{var} * \text{lift}_0(0)) = \text{lift}_{01}(\text{lift}_0(0) * \text{var})$
Interned Form	$\text{lift}_{10}(e_{92}) = \text{lift}_{01}(e_{13})$
Fresh Makeset	e_{99} (which we know is $\cdot \mapsto 0$)
Orient	$e_{92} \rightarrow \text{lift}_0(e_{99}), e_{13} \rightarrow \text{lift}_0(e_{99})$

E-matching Redundancies

- Non trivially thinned Union find edges
- Choices:
 - i. Just don't pattern match into those
 - ii. Choice 2:

- $x \mid - x * 0 = 0$

- $x, y \mid - 0 = ? \quad x, y \mid - ?a * ?b$

Two solutions:

1. $\{?a : x, ?b : 0\}$

2. $\{?a : y, ?b : 0\}$

Thinnings

- One *lifts* functions by *thinning* its arglist
- Thinnings are
 - A recipe to extract a subsequence
 - Representable as bitvectors (1 = keep, 0 = drop)
 - Graphically representable by non crossing wires from N to M dots
 - Strictly monotonic maps between total ordered sets
- Simpler than permutations
- Compaction of multiple de Bruijn shifts
- Have composition and identity

Connections

- McBride's Everybody's Got to Be Somewhere
 - $x, y \mapsto \sin(x)$
 - $n : \mathbb{N} \vdash (\text{Vec } n) \text{ Type}$
- Semi-Simpilical Categories
- Explicit Weakening Calculi
- Presheaves and Families

Differences with Slotted

Lifting	Slotted
Thinnings	(Partial) Permutations
Ordered Vars	Unordered Vars
Local Context	Global Context
Presheaf	Nominal

Similarities With Slotted

- Deal with "rigid" variables.
- `FatId` (renamed vs thinned)
- Special Union Finds
- Shape Computing Smart Constructors
- Public Slots ~ Minimal lifting
- Named vs Nameless is not really the difference

Lift Pulling Smart Constructors

```
class Node:  
    f : str  
    args : list[FatId]
```

- Smart enode construction $f(\text{lift}_i(X), \text{lift}_i(Y)) \rightarrow \text{lift}_i(f(X, Y))$
 - i. Find Largest Common Lifting of arguments (bitwise or)
 - ii. Factor it out of arguments
 - iii. Intern normalized enode
 - iv. Return eid with largest common lifting applied
- Thinnest context ~ free variables analysis
- Similar to McBride's Co-de Bruijn
- Alpha invariant Hash Cons

Comparing with Slotted

Why Do Lifts Appear?

- User wants a convenient context of 20 variables
- $x * 0 = 0$ makes lifts appear during rebuilding
- Children have different variables appearing in them

Binders

- Ok, but what about binders $\lambda, \forall, \exists, \int, \sum$?
- Binders are inessential. Lifting e-graphs make sense without them
- Binders have less dimensions than their children.
- Semantics: Binders are generalized higher order projection functions $(\mathbb{R}^d \rightarrow \mathbb{R}) \rightarrow (\mathbb{R}^{d-1} \rightarrow \mathbb{R})$
- Modified Pulling rules
 - "Transfer function" for liftings
 - Modified smart constructor
- Otherwise not much changes.
- `dump` and `subst` https://www.philipzucker.com/dump_calculus/

Substitution

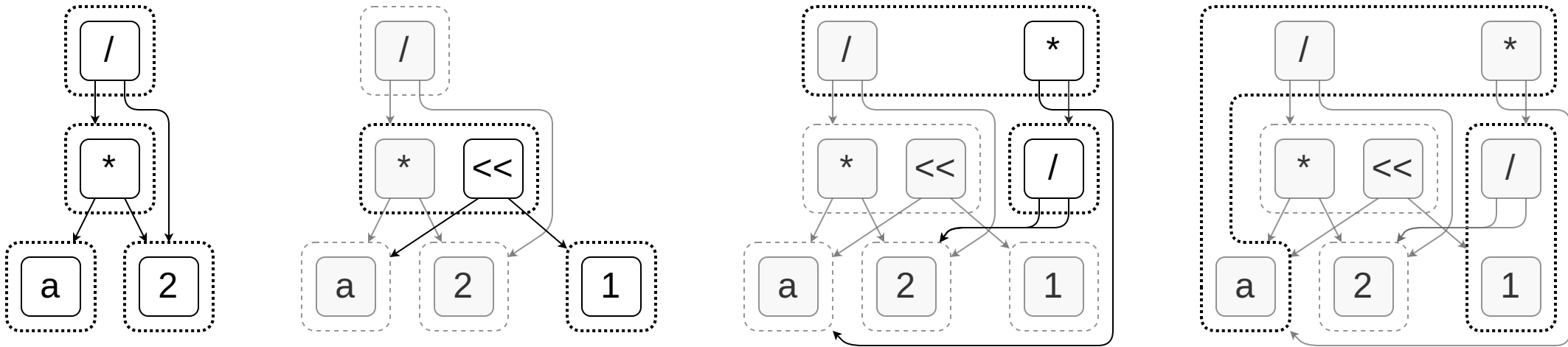
- β is hard and undecidable to bake in to e-matching
 - Higher order matching is undecidable
 - Restricted $\sim\alpha$ fragments like Miller β_0 are fine
- Bespoke substitution traversal routine may be useful. Thinnings tell when it may stop since $\sim$ free variable analysis
- Not sure

Rebuilding

- Reintern using smart constructor
- Not much changes

PreTalk : What are E-Graphs?

- Non destructive Rewriting and the Phase Ordering Problem
- A data structure for compactly storing trees / asts and an equality relation



PreTalk : What are E-Graphs?

- Hash Cons / Intern Table + Union Find
- Enables sharing parents rather than just children
 - Aliasing ordinarily allows share big subterms of `f(f(f(f(a))))` -> `g(f(f(f(a))))`
 - Naively, apply deep rewrite `a -> b` inside `f(f(f(f(a))))` -> `f(f(f(f(b))))` share no storage
 - Eclass indirection `f(f(f(f({a | b}))))` now can share
 - Bipartite graph of eclasses `{}` and enodes `f(_)`

Ordinary Union

- Example: $(x, y \mapsto y * 1) = (x, y \mapsto y)$
- Pulled: $\text{lift}_{01}(\text{var} * \text{lift}_0(1)) = \text{lift}_{01}(\text{var})$
- Union Call $\text{lift}_{01}(e_{17}) = \text{lift}_{01}(e_{42})$
- Peel off common lift to $e_{17} = e_{42}$
 - Semantics: lift_t is injective
- Pick parent $e_{17} \rightarrow e_{42}$

Redundancy Example 1: One Way Lift

- Example: $(x \mapsto x * 0) = (x \mapsto 0)$
- Pulled: $\text{var} * \text{lift}_0(0) = \text{lift}_0(0)$
- Union: $e_{92} = \text{lift}_0(e_8)$
- $e_{92} \rightarrow \text{lift}_0(e_8)$
 - Parent choice is forced

Redundancy Example 2: Incompatible liftings

- Example: $(x, y \mapsto x * 0) = (x, y \mapsto 0 * y)$
- Pulled: $\text{lift}_{10}(\text{var} * \text{lift}_0(0)) = \text{lift}_{01}(\text{lift}_0(0) * \text{var})$
- Interned: $\text{lift}_{10}(e_{92}) = \text{lift}_{01}(e_{13})$
- Makeset fresh parent e_{99}
 - Only relevant variables in intersection (bitwise and)
 - Semantics: Learned some directions are constant
- $e_{92} \rightarrow \text{lift}_0(e_{99})$, $e_{13} \rightarrow \text{lift}_0(e_{99})$

More

- Binders
- Baked in Substitution
- Types
- Generalized Union Finds

- Union

- i. Peel off common lift (vars both throw away)

- $\text{lift}_i(\text{lift}_j(e_1)) = \text{lift}_i(\text{lift}_k(e_2)) \implies \text{lift}_j(e_1) = \text{lift}_k(e_2)$

- lift_t is injective

- ii. Either

- $e_1 = e_2$ Ordinary union

- $e_1 = \text{lift}_i(e_2)$ Parent is forced.

- $\text{lift}_i(e_1) = \text{lift}_j(e_2)$ Makeset new parent

- Only relevant variables in intersection (bitwise and)

- Semantically we have learned some directions are constant (no actual dependency)

$$| \ x, y, z, w \mapsto x + z \mid \text{lift}_{1000}(\text{var}) + \text{lift}_{0010}(\text{var}) \mid \frac{\text{lift}_{1010}(\text{lift}_{10}(\text{var}) + \text{lift}_{01}(\text{var}))}{\text{lift}_{01}(\text{var})} \mid$$

$x^* \theta = \theta$ and Redundancies

Union

$x, y, \dots \mapsto _$	term
$(x, y \mapsto y * 1) = (x, y \mapsto y)$	$\text{lift}_{01}(\text{var} * \text{lift}_0(1)) = \text{lift}_{01}(\text{var})$
$(x \mapsto x * 0) = (x \mapsto 0)$	$\text{var} * \text{lift}_0(0) = \text{lift}_0(0)$
$(x, y \mapsto x * 0) = (x, y \mapsto y * 0)$	$\text{lift}_{10}(\text{var} * \text{lift}_0(0)) = \text{lift}_{01}(\text{var} * \text{lift}_0(0))$

$$\text{lift}_{01}(e_{y*1}) = \text{lift}_{01}(e_y)$$

Examples

$x, y, \dots \mapsto _$	Maximally Pulled Form
$x, y, z, w \mapsto 42$	$\text{lift}_{0000}(42)$
$x, y, z, w \mapsto 42 + 42$	$\text{lift}_{0000}(42 + 42)$
$x. y. z. w \mapsto z$	$\text{lift}_{0010}(\text{var})$
$x, y, z, w \mapsto z + 42$	$\text{lift}_{0010}(\text{var} + \text{lift}_0(42))$
$x, y, z, w \mapsto x + z$	$\text{lift}_{1010}(\text{lift}_{10}(\text{var}) + \text{lift}_{01}(\text{var}))$

Problems

1. sinsinsins(x) shares not data with sinsinsins(y)
2. variable generating rules
3. free variable analysis soundness
4. Names make me uneasy

Plot

$$x \mid - \triangleright \sin(x)$$

$x \mid - x$ is the identity

$[[x, y \mid - x]]$ is fst

$[[y, x \mid - x]]$ is snd

The context is a *part* of the term. It isn't "where" the term is.

